Revisión del diagrama V2

Proyecto Patitas Arriba · Documento de cambios y decisiones pendientes · 26/10/2026

Para quién es: el compañero que está corrigiendo el diagrama de base de datos (`diagrama-fisico_patitasV2.mwb`).

Qué contiene: los cambios que ya se aplicaron en el diagrama a partir de las correcciones que indicaste, las propuestas que se analizaron y se descartaron (con su razón) y las **4 decisiones que quedan en tus manos**.

Dónde está todo: en el repositorio, carpeta `Avances/sql/` : el `.mwb` actualizado, el DDL regenerado en minúscula (`patitasarriba-mysql.sql`) y este documento en versión editable (`cambios-diagrama.md`).

1. Resumen

#	Cambio o propuesta	Estado
1	El veterinario pasa de `cita` a `atencion_medica`	Hecho
2	Cobrar en la boleta los insumos usados en una atención	Hecho + pregunta abierta
3	Quitar `id_mascota` de `atencion_medica`	Hecho
4	Corregir la expresión de `subtotal` (columna calculada)	Hecho
5	`mascota.sexo` como `ENUM('MACHO','HEMBRA')`	Descartado
6	`nivel_gravedad` y `dia_semana` como `ENUM`	Hecho
7	Agregar `ESTETICO` y `EMERGENCIA` a los tipos de servicio	Decide el compañero
8	¿Se eliminan `tratamiento` y `atencion_tratamiento` ?	Decide el compañero
9	Borrado lógico (pedido de la JP) con las tablas de V2	Decide el compañero
10	`UNIQUE` sobre `id_detalle_cita` en `detalle_boleta_serv`	Descartado (lo controla el BL)
11	`UNIQUE` sobre `id_detalle_cita` en `atencion_medica`	Descartado (lo controla el BL)
12	Estado también en `detalle_cita`	Descartado

Decisiones que necesitamos de ti: (7) tipos de servicio, (8) qué pasa con `tratamiento` , (9) cómo se hace el borrado lógico, y la pregunta abierta del `UNIQUE` del cambio 2. Están en la sección 4, con una casilla para marcar tu elección.

2. La lógica de negocio que guía los cambios

Esta es la regla que explicaste en tus audios y que usamos como criterio en cada cambio: una **cita** es un contenedor donde el cliente pide varios servicios para su mascota; cada servicio pedido es un `detalle_cita` , y

la **atención** es la ejecución de ese servicio, que puede hacer un veterinario distinto en cada caso.

cita	**1 → n**	**detalle_cita**	**1 → 1**	**atencion_medica**	**1 → n**	**insumo_utilizado**
la reserva (una mascota)		cada servicio pedido		la ejecución, con su veterinario		lo que se usó (vacuna, algodón...)

Y para cobrar: la **boleta** reúne tres orígenes de líneas: los servicios (`detalle_boleta_serv`, vía `detalle_cita`), los insumos cobrables de las atenciones y las ventas directas de caja (los dos últimos en `detalle_boleta_art`).

3. Cambios ya aplicados y propuestas descartadas

3.1 Cambios aplicados en el diagrama

1. El veterinario pasa de `cita` a `atencion_medica`

Por qué: en una misma cita, cada atención puede hacerla un veterinario distinto (el baño con uno, el chequeo con otro). Con el veterinario en `cita` solo cabe uno por cita completa.

Qué se cambió:

- `cita`: se quitó `id_veterinario`, su índice y su FK. Quedan `id_cita`, `id_mascota`, `fecha_hora`, `estado` y `es_emergencia`.
- `atencion_medica`: se agregó `id_veterinario INT NOT NULL` con FK a `veterinario`.

Decisiones: es obligatorio en la atención (una atención ejecutada siempre tiene quién la hizo); `horario_veterinario` no cambia.

3. Quitar `id_mascota` de `atencion_medica`

Por qué: es redundante. La mascota ya se obtiene por `detalle_cita → cita → mascota`. Tener las dos rutas permitía datos contradictorios (una atención de una mascota enlazada a la cita de otra). Toda atención tiene `id_detalle_cita NOT NULL`, así que no existen atenciones sueltas que necesiten una mascota propia.

Qué se cambió: se quitó `id_mascota`, su FK y su índice. Ahora solo `cita` apunta a `mascota`.

2. Cobrar en la boleta los insumos usados en una atención

El problema: en una atención se usan insumos (vacuna, algodón, alcohol). Algunos los **paga el cliente aparte** (la vacuna) y otros **ya vienen incluidos** en el precio del servicio (el algodón). Hasta ahora `insumo_utilizado` registraba todo, pero la boleta solo sabía cobrar servicios y ventas de caja, así que la vacuna usada en la atención no aparecía en ninguna parte.

La solución (dos arreglos que van juntos):

- **A. Distinguir qué se cobra:** campo `es_cobrable` en `insumo_utilizado` (1 = lo paga el cliente aparte, 0 = incluido en el servicio).
- **B. Dónde se cobra:** campo opcional `id_insumo_utilizado` en `detalle_boleta_art`, con FK a `insumo_utilizado`. Vacío = venta directa de caja; con valor = insumo de una atención que se está cobrando.

Tabla	Cambio	Para qué

`insumo_utilizado`	+ `es_cobrable TINYINT(1) NOT NULL DEFAULT 0`	saber qué insumos se cobran
`detalle_boleta_art`	+ `id_insumo_utilizado INT NULL` con FK	enlazar la línea con el insumo
`detalle_boleta_art`	+ `UNIQUE (id_insumo_utilizado)`	que un insumo no se cobre dos veces
`detalle_boleta_art`	– índice único `(id_boleta, id_articulo)`	permitir dos líneas del mismo artículo si vienen de insumos distintos

Por qué sale el índice `(id_boleta, id_articulo)`: antes obligaba a una sola línea por artículo con la cantidad dentro. Con los insumos, el mismo artículo puede salir de dos atenciones distintas de una misma boleta (dos vacunas en dos servicios), y para saber de qué atención salió cada cobro hacen falta dos líneas. La boleta que ve el cliente puede mostrarse agrupada; la trazabilidad interna queda por línea.

Redundancia aceptada: la línea guarda `id_articulo` y `id_insumo_utilizado` (el insumo ya conoce su artículo). Se acepta a cambio de no tocar las ventas directas; el BL validará que ambos coincidan.

Ejemplo (una sola boleta, varias líneas). Ana lleva a Firulais: vacunación (S/ 50) y consulta (S/ 80). Se usó una vacuna cobrable (S/ 35), 2 desparasitantes cobrables (S/ 10 c/u), algodón y alcohol (incluidos), y Ana compra un shampoo (S/ 25) en caja.

Línea	Tabla	Subtotal
Vacunación	`detalle_boleta_serv`	S/ 50
Consulta médica	`detalle_boleta_serv`	S/ 80
Vacuna antirrábica (insumo 101)	`detalle_boleta_art`	S/ 35
Desparasitante × 2 (insumo 104)	`detalle_boleta_art`	S/ 20
Shampoo (venta directa)	`detalle_boleta_art`	S/ 25
Total de la boleta		**S/ 210**

El algodón y el alcohol no se cobran porque su `es_cobrable` es 0. Es una sola boleta; las líneas solo se guardan en dos tablas distintas.

Cómo se arma la boleta: (1) una línea de servicio por cada `detalle_cita`; (2) se agregan los insumos con `es_cobrable = 1` que aún no estén en ninguna boleta; (3) se suman las ventas directas de caja; (4) el total es la suma de los subtotales.

Importante para el Java: hoy registrar un insumo en la atención **no descuenta stock**. Lo coherente es descontarlo en la atención (sea cobrable o no, porque físicamente se consumió) y que las líneas de boleta con `id_insumo_utilizado` no lo descuenten otra vez.

4. Corregir la expresión de `subtotal`

Por qué: en `detalle_boleta_art` y `detalle_boleta_serv`, `subtotal` es una columna que MySQL calcula sola (`cantidad × precio_unitario`). La fórmula estaba guardada con acentos graves (backticks) de más, y al generar el DDL salía SQL inválido que MySQL rechazaba. Había que corregirlo a mano cada vez que se regenerara el script.

Qué se cambió: en las dos tablas la expresión es ahora `CANTIDAD * PRECIO_UNITARIO`, sin backticks, con almacenamiento `STORED` en ambas.

Para el Java: los procedures de `detalle_boleta_*` no deben insertar ni actualizar `subtotal`; solo se lee.

6. `nivel_gravedad` y `dia_semana` pasan de `VARCHAR` a `ENUM`

Por qué: en Java ya son enums (`NivelGravedad`, `DiaSemana`) y los DAOs los guardan con `name()` y los leen con `valueOf()`. Con `VARCHAR`, la base aceptaba cualquier texto y un valor inválido solo fallaba después, al leerlo. Con `ENUM` la base lo rechaza y el diagrama documenta los valores permitidos, igual que en los demás ENUM.

- `atencion_diagnostico.nivel_gravedad` → `ENUM('LEVE','MODERADO','GRAVE','CRITICO')`, sigue siendo opcional.
- `horario.dia_semana` → `ENUM('LUNES','MARTES','MIERCOLES','JUEVES','VIERNES','SABADO','DOMINGO')`, sigue siendo obligatorio.

Para el Java: nada que cambiar.

3.2 Propuestas analizadas y descartadas

5. `mascota.sexo` como `ENUM('MACHO','HEMBRA')` → se queda `ENUM('M','H')`

Por qué se propuso: uniformidad con los demás ENUM, cuyo valor coincide con el nombre del enum de Java.

Por qué se descarta: el diseño anterior ya usaba `'M'` y `'H'`, y `MascotaDAOImpl` ya traduce `MACHO ⇔ 'M'` y `HEMBRA ⇔ 'H'` correctamente (el Tester de Mascota pasó). Cambiarlo obligaba a tocar procedure y DAO sin ganar nada funcional.

10. `UNIQUE (id_detalle_cita)` en `detalle_boleta_serv` → lo controla el BL

Qué buscaba: que un servicio de una cita no se cobre dos veces.

Por qué se descarta: el `UNIQUE` también bloquearía un caso normal. Si una boleta se **anula** y hay que emitir otra corrigiendo un error (por ejemplo, el método de pago), la línea de la boleta anulada sigue en la tabla y el `UNIQUE` impediría volver a cobrar ese servicio. MySQL no permite un `UNIQUE` que valga solo para boletas no anuladas.

Decisión: el BL comprueba, antes de agregar la línea, que el `detalle_cita` no esté en una boleta no anulada.

11. `UNIQUE (id_detalle_cita)` en `atencion_medica` → lo controla el BL

Qué buscaba: reflejar la regla "cada servicio pedido se ejecuta en una sola atención" (relación 1 a 1).

Por qué se descarta: mismo problema que el 10. Si una atención se anula por un error de registro, hay que registrarla otra vez, pero la anulada seguiría ocupando ese `id_detalle_cita`.

Decisión: el BL comprueba, antes de registrar una atención, que el `detalle_cita` no tenga otra atención no anulada.

12. Estado también en `detalle_cita` → no se agrega

Qué buscaba: reflejar que los servicios de una cita se ejecutan por separado (el baño puede estar hecho y la vacuna pendiente).

Por qué se descarta: el estado de cada servicio se puede **deducir** de lo que ya existe:

- *Pendiente:* el `detalle_cita` no tiene ninguna atención con `anulado = 0`.
- *Ejecutado:* tiene una atención con `anulado = 0`.
- *Cancelado individualmente:* su `activo = 0` (si se aprueba la propuesta 9).

La cita pasa a `COMPLETADO` cuando todos sus servicios activos están ejecutados, y eso lo calcula el BL al registrar la última atención. `APROBADO` y `RECHAZADO` son decisiones sobre la cita completa. Una columna extra duplicaría información de `atencion_medica` y podría desincronizarse.

4. Decisiones que quedan en tus manos

Para cada una dejamos la situación, las opciones, nuestra recomendación y una casilla para marcar lo que decidas.

Decisión A (propuesta 7): tipos de servicio

Situación: en V2, `servicio.tipo_servicio_medico` solo admite `CONSULTA_MEDICA`, `OPERACION` y `VACUNACION`. El Java (`TipoServicio`) tiene además `EMERGENCIA` y `ESTETICO`. Si se intenta guardar un servicio de esos dos tipos, MySQL responde *Data truncated*. V2 parece venir de una versión anterior a una corrección que ya se hizo en `ServicioDAOImpl`.

Recomendación: ampliar el ENUM a `('CONSULTA_MEDICA','OPERACION','VACUNACION','EMERGENCIA','ESTETICO')`. `ESTETICO` es necesario (un baño es un servicio estético y lo mencionaste como ejemplo de servicio de una cita). `EMERGENCIA` es opcional porque ahora existe `cita.es_emergencia`, pero conviene dejarlo para que la base y el Java coincidan sin tocar código. Costo en Java y procedures: ninguno.

- ☐ Agregar los dos (`ESTETICO` y `EMERGENCIA`)
- ☐ Agregar solo `ESTETICO` (y quitar `EMERGENCIA` del Java)
- ☐ Dejarlo como está en V2

Decisión B (propuesta 8): ¿se eliminan `tratamiento` y `atencion_tratamiento`?

Situación: V2 no tiene esas dos tablas, y `insumo_utilizado` ya no cuelga de `atencion_tratamiento` sino directo de `atencion_medica`. No se mencionó en los audios; parece una decisión de diseño, pero no está confirmada.

Qué eran en el Java: `Tratamiento` es un catálogo de procedimientos (`nombreProcedimiento`, `descripcion`); `AtencionTratamiento` registra qué tratamientos se aplicaron en una atención, con sus insumos.

Por qué podría ser intencional: si la atención es la ejecución de un servicio, y el servicio ya dice qué se hace (consulta, operación, vacunación, baño), un catálogo aparte de tratamientos duplica la idea.

Qué arrastra si se eliminan (~14 archivos de Java): modelo (`Tratamiento`, `AtencionTratamiento` y sus referencias en `AtencionMedica`); persistencia (`TratamientoDAO` e `Impl`, `AtencionTratamientoDAO` e `Impl`, y referencias en `AtencionMedicaDAOImpl` e `InsumoUtilizadoDAOImpl`); negocio (`TratamientoBL` e `Impl`); SQL (`procedures-tratamiento.sql`, `procedures-atencion_tratamiento.sql` y una línea de `limpiar-tablas.sql`); app (`probarTratamiento` del Tester y referencias en `Principal`).

Recomendación: opción A (eliminarlas), coherente con V2 y con la lógica que explicaste, pero confirmándolo con el grupo y con los dueños de esos módulos antes de borrar código. Conviene revisar antes el catálogo de requerimientos por si menciona tratamientos como funcionalidad pedida.

- ☐ **A.** Se eliminan (cada dueño retira su Java)
- ☐ **B.** Se restituyen al diagrama y se reconecta `insumo_utilizado`

Decisión C (propuesta 9): borrado lógico con las tablas de V2

El problema: la JP pidió que **todos los deletes sean lógicos** (marcar el registro como inactivo en lugar de borrarlo). V2 dejó **12 tablas sin columna `activo`**, pero **todas las clases del modelo Java extienden `Registro`**, que trae `id` y `activo`. Esas tablas no se pueden desactivar y sus clases fallarían al leer o insertar buscando una columna que no existe.

Estado de las 23 tablas en V2:

Grupo	Tablas	Cómo se "elimina"
Con `activo` (11)	`administrador`, `articulo`, `categoria`, `cliente`, `cuenta`, `diagnostico`, `horario`, `mascota`, `receta`, `servicio`, `veterinario`	`UPDATE ... SET activo = 0`
Con indicador propio (3)	`boleta`, `atencion_medica` (`anulado`); `cita` (`estado`)	`anulado = 1` / `estado = 'CANCELADO'`
Sin ningún indicador (9)	`detalle_cita`, `detalle_receta`, `detalle_boleta_art`, `detalle_boleta_serv`, `atencion_diagnostico`, `insumo_utilizado`, `articulo_categoria`, `horario_administrador`, `horario_veterinario`	por definir

Cómo se elimina hoy en el código: es el patrón maestro–detalle que mostró el profesor: al eliminar un maestro, **primero se eliminan sus detalles y al final el maestro, en una sola transacción manejada desde el BL**. Ya existe en `CitaDAOImpl.delete` (detalles de la cita y luego la cita) y en `eliminar_detalles_por_receta` (que pone `activo = 0` en los detalles de la receta). Es además lo que demuestra el requisito de la rúbrica sobre transacciones.

Por qué choca con V2: con borrado lógico la cascada también debe ser lógica (marcar los detalles como inactivos y luego el maestro). Pero si los detalles no tienen `activo`, la única forma de "eliminarlos" es borrarlos de verdad mientras el maestro queda ahí inactivo, y eso destruye el historial: una boleta anulada se quedaría sin sus líneas, una cita cancelada sin sus servicios, una atención anulada sin diagnósticos ni insumos.

Propuesta:

- **1. Agregar `activo TINYINT(1) NOT NULL DEFAULT 1` a las tablas detalle que cuelgan de un maestro:** `detalle_cita`, `detalle_receta`, `detalle_boleta_art`, `detalle_boleta_serv`, `atencion_diagnostico` e `insumo_utilizado`. *Porqué:* la cascada lógica necesita marcar cada línea como inactiva; sin la columna solo se puede borrar la línea y se pierde el historial.
- **2. Eliminar = cascada lógica en una sola transacción desde el BL:** primero `activo = 0` en los detalles; luego `activo = 0`, `anulado = 1` o `estado = 'CANCELADO'` en el maestro. *Porqué:* es el patrón que ya usa el proyecto, y la transacción evita hijos activos con padre desactivado si algo falla a la mitad.
- **3. Los enlaces puros quedan sin `activo` y se borran con `DELETE`:** `articulo_categoria`, `horario_administrador`, `horario_veterinario`. *Porqué:* solo dicen "este artículo está en esta categoría" o "este veterinario tiene este horario"; no tienen historia propia y borrarlos no pierde nada.
- **4. Los maestros con indicador propio lo conservan:** `boleta` y `atencion_medica` con `anulado = 1`, `cita` con `estado = 'CANCELADO'`. *Porqué:* anular una boleta o una atención y cancelar una cita expresan mejor el negocio que un `activo = 0`; esos registros no se eliminan, se invalidan.

La decisión más importante para el Java: `boleta`, `atencion_medica` y `cita` no tienen `activo`, pero sus clases heredan de `Registro`.

☐ **A.** Agregar también `activo` a esas tres tablas (el Java queda sin cambios, pero habría `activo` y `anulado` / `estado` a la vez, algo redundante).

☐ **B.** Cambiar la jerarquía en Java para que `Registro` no obligue a `activo` en esas tres clases (más limpio, pero toca clases de varios módulos).

☐ Apruebo agregar `activo` a las 6 tablas detalle (puntos 1 a 4 de la propuesta)

☐ Prefiero otra solución: _____

Una vez decidido, los procedures `eliminar_*` de maestros y detalles pasan a `UPDATE`; los de los detalles necesitan la columna `activo`; los enlaces puros siguen con `DELETE`.

Decisión D (pregunta abierta del cambio 2): el `UNIQUE (id_insumo_utilizado)`

Situación: el `UNIQUE` sobre `id_insumo_utilizado` en `detalle_boleta_art` impide cobrar un insumo dos veces, pero tiene el mismo efecto secundario que descartamos en las propuestas 10 y 11: también impediría **volver a cobrarlo después de anular una boleta**, porque la línea de la boleta anulada sigue ocupando ese `id_insumo_utilizado`.

Recomendación: quitarlo y que el BL compruebe, antes de cobrar un insumo, que no esté en una boleta no anulada. Es coherente con lo decidido para `detalle_boleta_serv` y `atencion_medica`.

☐ Quitar el `UNIQUE` y controlarlo en el BL

☐ Dejar el `UNIQUE` (re-facturar un insumo tras anular exigiría un procedimiento aparte que libere la línea anulada)

5. Qué pasa después de tus decisiones

1. Se aplican en el `.mwb` los cambios que apruebes (una sola persona edita el diagrama: es un archivo binario y git no puede fusionar dos versiones).
2. Se vuelve a generar el DDL con *Forward Engineer* y se pasa a minúscula (los ENUM se quedan en mayúscula).
3. Cada quien actualiza los procedures y el Java de su módulo a partir del modelo, y el Tester se corrige al final.
4. Cuando cambie el DDL, quien ya tenga la base creada debe hacer `DROP DATABASE mydb;` antes de volver a ejecutarlo (`CREATE TABLE IF NOT EXISTS` no modifica tablas existentes, y `cuenta` y `CUENTA` son tablas distintas en un servidor Linux como AWS).

Impacto en el código de los cambios ya aplicados, para cuando el diagrama esté final:

Cambio	Qué se adapta
1 y 3 (veterinario y mascota)	`Cita` pierde al veterinario (modelo, DAO, BL, procedures y Tester); `AtencionMedica` gana al veterinario y obtiene la mascota por la cita; los procedures de búsqueda por mascota necesitarán un `JOIN`.
2 (insumos y boleta)	El stock se descuenta al registrar el insumo en la atención; `DetalleBoleta` gana un `insumoUtilizado` opcional; el BL valida que el artículo coincida con el del insumo.
4 (`subtotal`)	Los procedures de `detalle_boleta_*` no insertan ni actualizan `subtotal`.
6 (ENUM)	Nada.

Pendiente al final: pasar a minúscula los nombres dentro del propio `.mwb` (el diagrama sigue con mayúsculas; el DDL ya sale en minúscula).